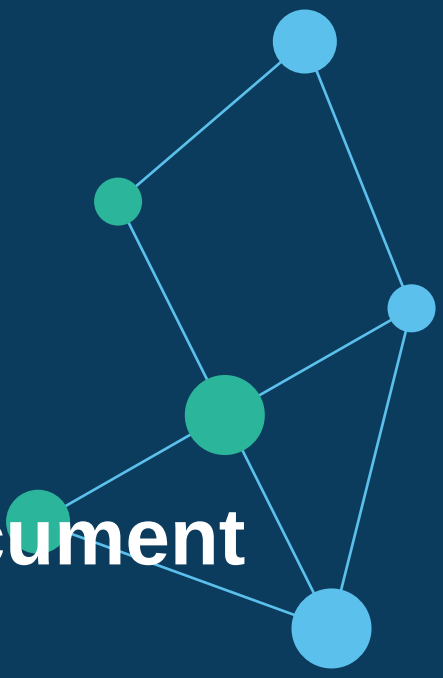


Application Design Document

AI-Powered Telecom Network Operations Assistant

Multi-agent decision support for historical incident retrieval, root-cause hypotheses, network impact, prediction and SOP-backed troubleshooting recommendations.

Table of Contents

1. Document Information 4

1.1 Purpose . . . 4

1.2 Scope . . . 4

1.3 Audience . . . 4

1.4 Evidence and trust conventions . . . 4

2. Executive Summary 4

3. Functional Requirements 5

4. Non-Functional Requirements 6

5. Technical Stack and Tools 6

5.1 Stack by layer . . . 7

5.2 Services and ports . . . 7

5.3 LLM usage and credentials . . . 7

5.4 Main dependencies . . . 7

6. Architecture 8

6.1 Reading the architecture . . . 9

6.2 Component boundaries . . . 9

7. End-to-End Request Sequence 9

7.1 Stage inputs and outputs . . . 10

8. Agent Responsibility Matrix 11

9. Retrieval and Embedding Design 11

9.1 Limitations . . . 11

10. Prediction Design 12

10.1 Feature groups . . . 12

10.2 Validation and missing-feature behaviour . . . 12

10.3 Interpretation . . . 12

11. Synapt MCP Design 12

12. API Design 13

12.1 GET /api/health . . . 13

12.2 POST /api/analyze . . . 13

12.3 POST /api/predict . . . 13

12.4 POST /api/classify . . . 13

12.5 POST /api/feedback . . . 14

13. User Interface Design 14

14. Data Design 14

15. Security Design 15

16. Evaluation Design 15

16.1 Objectives and judge . . . 15

16.2 Metrics . . . 15
16.3 Current results . . . 15
16.4 Limitations and interpretation . . . 16
17. Failure Handling and Fallback Matrix 16
18. Deployment and Runbook 17
18.1 Health checks . . . 17
19. Risks, Limitations and Future Improvements 17
20. Appendix 18
A. Repository file map . . . 18
B. Environment variables (names only) . . . 18
C. Sample queries . . . 18
D. Glossary . . . 19
21. References and Source Files 19

1. Document Information

Item	Detail
Document	Application Design Document
Project	AI-Powered Telecom Network Operations Assistant (telecom network decision-support assistant)
Version / status	1.0 - capstone submission draft for technical review
Date	11 October 2026
Prepared by	Project team
System status	Working prototype. Not connected to live telecom network telemetry.

1.1 Purpose

This document describes the design of the implemented prototype: its requirements, technology stack, architecture, agent responsibilities, data, retrieval, prediction, Synapt integration, APIs, user interface, security, evaluation and operation. It is written for a capstone final submission and a technical review. Where a capability is optional, legacy, synthetic or not production-ready, it is labelled as such.

1.2 Scope

In scope	Out of scope
Natural-language incident analysis over a historical dataset; Synapt domain context and SOP retrieval; hybrid retrieval; RCA hypotheses; incident classification; potential-impact estimates; historical KPI averages; optional live prediction from supplied telemetry; feedback capture; DeepEval and RAGAS evaluation; local deployment.	Live telecom network integration, real-time telemetry ingestion, production monitoring, automatic model retraining, multi-tenant deployment, and validation of Synapt SOP content against approved operational procedures.

1.3 Audience

Project reviewers and examiners, the engineering team that maintains the system, and telecom operations stakeholders who evaluate whether the approach could support network-engineer triage.

1.4 Evidence and trust conventions

The system keeps four kinds of output separate, and this document uses the same labels throughout:

Output	How it must be read
Historical evidence	Records retrieved from the incident dataset. This is the only evidence source for RCA.
RCA result	A ranked, evidence-linked hypothesis. It is not a confirmed physical root cause.
Prediction	A model estimate from supplied telemetry. It is not a live observation.
Historical baseline	A descriptive average over filtered historical records. It is not live telemetry or a forecast.
Impact	Potential network or service effects. It is not a confirmed customer outcome.
Recommendation	Procedure-based suggestions that require validation against approved operational procedures.

2. Executive Summary

Problem. Telecom networks regularly suffer congestion, high latency, packet loss, dropped calls, handover failures and service degradation. Historical records hold the metrics, alarms, configuration changes and past faults needed to understand such events, but searching them by hand to find similar incidents is slow, and engineers also need likely causes, risk assessment and recommended actions.

Solution. The application turns a natural-language network question into structured analysis. After an input guardrail, a Query agent plans the query, obtains telecom domain context from Synapt, filters the historical dataset and retrieves similar incidents using hybrid semantic and lexical search. Downstream agents produce evidence-linked

RCA hypotheses, incident classification and potential-impact estimates. When valid live telemetry is supplied, two scikit-learn models estimate throughput and fault occurrence. When troubleshooting is requested, a Recommendation agent retrieves SOP procedures through Synapt. A React interface renders the answer with its evidence and trace.

Key capabilities

- Guardrailed natural-language queries over a 100,000-row, 27-column historical incident dataset.
- Synapt Context Substrate integration through MCP for domain/schema context and on-demand SOP procedures.
- Hybrid retrieval: local BAAI/bge-small-en-v1.5 embeddings in ChromaDB combined with BM25.
- RCA hypotheses, incident classification and impact estimates that state their uncertainty.
- Historical KPI averages clearly separated from live model predictions.
- Feedback capture (SQLite) and DeepEval/RAGAS evaluation with saved reports.

Trust statement: RCA hypotheses are not confirmed causes. Impact findings are not confirmed customer outcomes. Historical baselines are not live telemetry or forecasts. Predictions are estimates. Recommendations require operational validation.

3. Functional Requirements

ID	Requirement	Realised by	Label / constraint
FR-01	Validate and sanitise natural-language input; block unsafe or irrelevant queries.	InputGuardrail	Blocked queries get a safe response and the workflow stops.
FR-02	Search historical incidents from a natural-language query with metadata and explicit numeric filters.	QuerySynaptAgent, query plan	Numeric conditions only when the user states them.
FR-03	Retrieve telecom domain and schema context from Synapt.	Synapt MCP ask_synapt	Advisory only. Not historical evidence.
FR-04	Rank incidents with hybrid semantic + lexical retrieval.	ChromaDB, BM25, hybrid ranking	Evidence for RCA.
FR-05	Generate ranked, evidence-linked root-cause hypotheses.	RCAAgent	Hypothesis, not confirmed cause.
FR-06	Classify incident priority and the current scenario.	IncidentClassificationAgent	Dataset-relative, prototype rules. Not SLAs or 3GPP limits.
FR-07	Estimate potential network and service impact.	NetworkImpactAnalyzer	Potential, not confirmed customer impact.
FR-08	Report historical KPI averages for filtered records on request.	Historical evaluation agent	Baseline, never a live prediction.
FR-09	Predict throughput and fault occurrence from supplied live telemetry.	NetworkPredictionAgent, joblib models	Runs only with the complete required feature set.
FR-10	Retrieve SOP/procedure records when troubleshooting is requested.	Synapt get_synapt_procedures	Status: success, no_procedures, unavailable.
FR-11	Turn procedures into recommended actions.	RecommendationAgent	Requires operational validation.
FR-12	Capture helpful / not-helpful feedback for later evaluation.	POST /api/feedback, SQLite	Stored in feedback/feedback.sqlite3.
FR-13	Evaluate answers, retrieval and RCA, and display saved reports.	DeepEval, RAGAS, Evaluations page	Reports in evaluations/reports/.

4. Non-Functional Requirements

Category	Requirement	Design response
Latency	Interactive analysis for a single query.	Local embeddings and BM25 avoid network round trips. LLM calls (planning, summary) dominate variability; timeout, temperature, token limit and retries are configurable in llm/llm_client.py. No formal latency target was supplied, and none is claimed.
Reliability	Partial failure must not break the response.	Conditional stages, deterministic fallbacks and explicit status fields (Section 17).
Explainability	Every conclusion shows its basis.	Historical evidence, evidence strength, contradictory or missing evidence, and trace metadata accompany the answer.
Security	No secret exposure; safe handling of untrusted input.	Credentials only in .env files, guardrail and Pydantic validation, localhost binding (Section 15).
Privacy	Avoid unnecessary personal data.	The dataset is network telemetry and incident data. Queries and feedback stay local (SQLite), and prompts to the external LLM carry the query and retrieved evidence.
Maintainability	Clear module boundaries.	One agent per module, LangGraph orchestration, canonical metric schema in config/metric_schema.py, prediction features in config/prediction_config.py.
Local-service boundaries	Components communicate over defined local ports.	Synapt MCP 8000, API 8001, frontend 8790. No component is exposed beyond localhost by default.
Graceful fallback	Degrade rather than fail.	Deterministic query parser, deterministic summary, no-context retrieval, incomplete-prediction status.

5. Technical Stack and Tools

Frontend: the user interface is built only with React 18.3 and Vite 5.4 (JavaScript/JSX and CSS). Streamlit is not used.

5.1 Stack by layer

Layer	Technology	Role in the system
Language / runtime	Python 3.10+	Backend, agents, retrieval, prediction and evaluation tooling.
Frontend	React 18.3, Vite 5.4, JSX, CSS	Login, AI Assistant, ML prediction, scenario comparison and evaluation pages.
API	Starlette (ASGI) served by Uvicorn	HTTP endpoints and orchestration entry points.
Validation	Pydantic 2.x	Request models and input validation.
Orchestration	LangGraph (agents/orchestrator.py)	State graph that carries a structured AnalysisState between stages.
LLM	AICredits OpenAI-compatible chat API; google/gemini-3.1-flash-lite for evaluation; langchain-openai	Query planning, response summarisation, LLM-as-judge.
Embeddings	sentence-transformers with BAAI/bge-small-en-v1.5 (local)	Semantic vectors for incident text.
Vector store	ChromaDB (persistent local collection)	Semantic similarity search.
Lexical search	rank-bm25 (BM25)	Keyword relevance.
Data	pandas, NumPy	Dataset loading, filtering and descriptive averages.
Machine learning	scikit-learn, joblib	Throughput and fault-occurrence models stored as .joblib artifacts.
Domain integration	MCP (Python mcp package), httpx / requests, Synapt	Domain context and SOP retrieval through the local MCP bridge.
Evaluation	DeepEval, RAGAS	LLM-as-judge metrics and RAG metrics.
Feedback storage	SQLite	feedback/feedback.sqlite3.
Configuration	python-dotenv, .env files	Credentials and provider settings.

5.2 Services and ports

Service	Port	Notes
Synapt MCP bridge	8000	Endpoint http://127.0.0.1:8000/mcp. Domain context and SOP retrieval.
Backend API (Starlette/Uvicorn)	8001	All /api/* endpoints.
React frontend	8790	Open http://127.0.0.1:8790.

5.3 LLM usage and credentials

- **Active provider:** AICredits (OpenAI-compatible). The model is set through DEEPEVAL_MODEL and the AICredits model settings. The current evaluation model is google/gemini-3.1-flash-lite.
- **LLM client:** llm/llm_client.py provides provider selection, timeout, temperature, token limit and retry settings.
- **LLM uses:** query planning, response summarisation and LLM-as-judge evaluation. Classification, impact analysis and prediction do not depend on an LLM.
- **Credentials:** the root .env holds the AICredits key. synapt_mcp_bundle/.env holds Synapt provider and agent credentials. Values are never shown in this document.
- **Legacy:** OpenRouter and Claude settings remain in code and environment files for compatibility but are not the active provider.

5.4 Main dependencies

pandas, numpy, scikit-learn, joblib, pydantic, sentence-transformers, chromadb, rank-bm25, requests, python-dotenv, httpx, mcp, deepeval, ragas, langchain-openai, react, react-dom, and vite.

6. Architecture

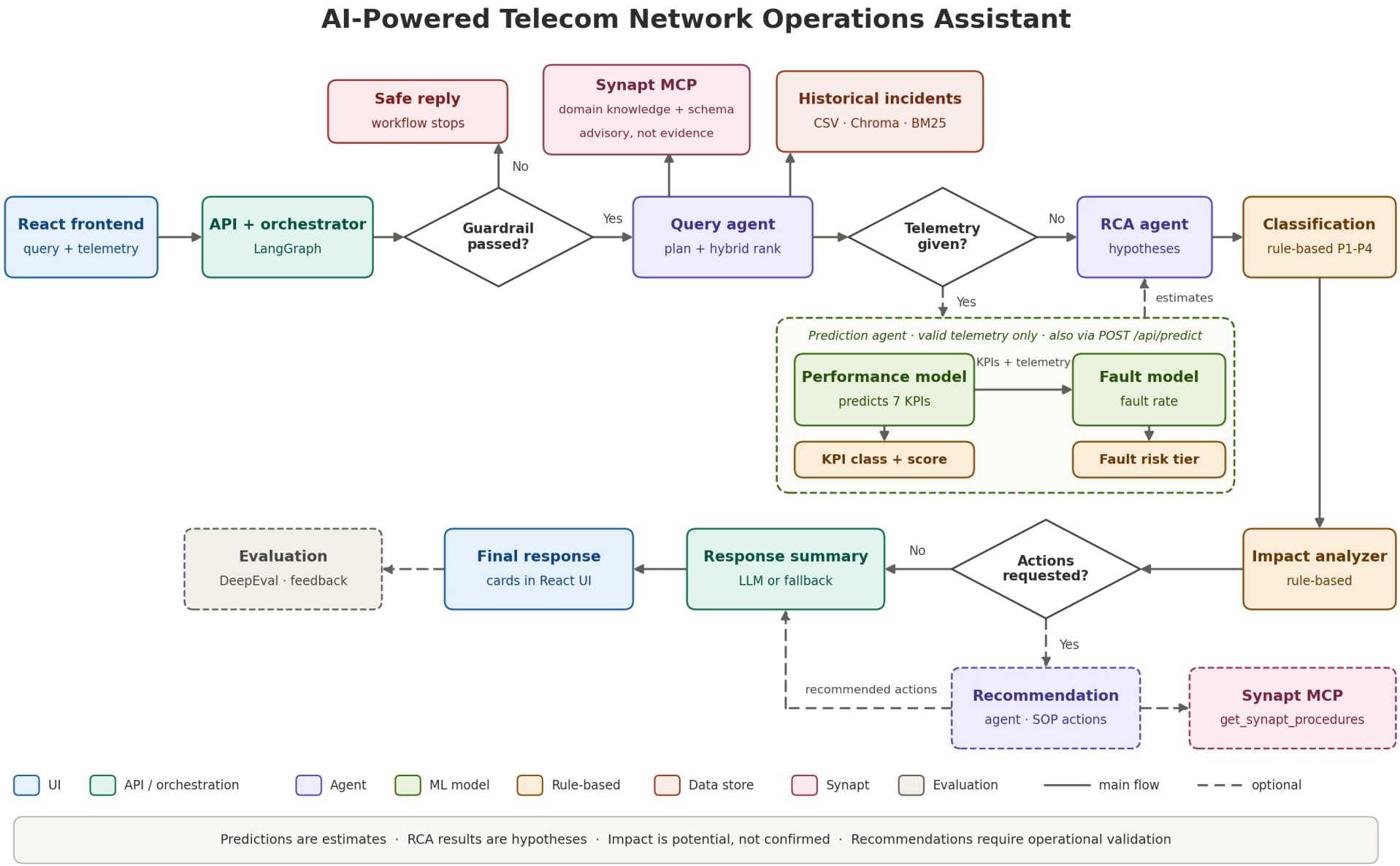


Figure 1. System architecture of the AI-Powered Telecom Network Operations Assistant. Solid arrows show the main flow; dashed arrows show optional branches (live prediction, recommendations, SOP retrieval, evaluation).

6.1 Reading the architecture

Figure 1 is the authoritative architecture for this project. Where a label in the figure and Section 10 differ on the exact prediction targets, Section 10 (based on config/prediction_config.py) takes precedence. The figure shows five groups of components:

- **Frontend (React).** Collects the query and optional telemetry, calls the API, and renders answer cards, evidence, trace and the evaluation page.
- **API (Starlette).** Validates HTTP requests with Pydantic and hands analysis to the orchestrator.
- **Orchestrator (LangGraph).** Runs the stages in order, passes a structured AnalysisState, and applies the conditional branches: guardrail approval, telemetry availability, recommendation request and summariser availability.
- **Retrieval and Synapt.** The Query agent obtains advisory context from Synapt, plans the query and searches the historical incident dataset using ChromaDB and BM25.
- **Analysis, prediction and recommendation agents.** RCA, classification and impact consume the retrieved evidence. Prediction and recommendation are optional branches.

6.2 Component boundaries

Component	Owns	Does not do
Frontend	Input forms, rendering, client-side checks, feedback buttons, saved-report display.	No analysis logic, no direct LLM or Synapt calls.
API	HTTP validation, routing, serialisation, CORS.	No retrieval or modelling logic.
Orchestrator	Stage order, shared state, branch decisions, response assembly, trace metadata.	No domain computation inside the graph itself.
Retrieval	Schema normalisation, filters, embeddings, Chroma and BM25 search, hybrid ranking.	Does not interpret causes or recommend actions.
Synapt	Domain/schema context and SOP/procedure records through MCP.	Context is not evidence and does not prove root cause. SOPs are not historical incidents.
Prediction	Validates required telemetry, runs the joblib models, maps fault risk.	Does not run without the full required feature set. Never turns averages into predictions.
RCA	Ranks hypotheses using retrieved incidents, telemetry and predictions.	Does not claim confirmed causes.
Impact	Rule-based potential network and service effects.	Does not state confirmed customer impact.
Recommendation	Calls Synapt procedure retrieval and converts procedures to actions.	Runs only when requested. Does not invent procedures.
Evaluation	DeepEval and RAGAS scoring, saved reports, feedback review.	Not part of the live request path.

Three sources are kept strictly separate: (1) historical incident retrieval from the CSV through Chroma and BM25, (2) Synapt domain-context retrieval (ask_synapt), and (3) Synapt SOP/procedure retrieval (get_synapt_procedures).

7. End-to-End Request Sequence

1. **React query form.** The user submits a natural-language query, with optional current telemetry, on the AI Assistant page.
2. **POST /api/analyze.** The API validates the request and calls TelecomAnalysisOrchestrator.
3. **Input guardrail.** The query is checked and sanitised. A blocked or irrelevant query ends the workflow with a safe response.
4. **Query planning.** An LLM plans the query. If the planner fails, a deterministic parser produces the plan. Metrics are normalised to the canonical schema.
5. **Synapt context.** The Query agent calls ask_synapt for advisory telecom domain context.
6. **Retrieval.** Metadata and explicit numeric filters are applied, then Chroma semantic search and BM25 run and are combined into a ranked list of historical incidents.

7. **Historical baseline (when asked).** For queries about averages, performance, trends, assessment or impact, the historical evaluation agent computes descriptive averages over the filtered records.
8. **Live prediction (optional).** Only if complete, valid telemetry is supplied, the Prediction agent runs the models. Otherwise this stage is skipped.
9. **RCA.** The RCA agent ranks evidence-linked hypotheses using incidents, the Query handoff, and telemetry or predictions if present.
10. **Classification.** The incident classifier assigns historical priority (P1 to P4) and, with telemetry, a current-scenario class.
11. **Impact.** The impact analyzer estimates potential effects from telemetry, predictions or historical averages.
12. **SOP retrieval (optional).** If troubleshooting, recommendations, SOPs or resolution steps were requested, the Recommendation agent calls get_synapt_procedures.
13. **Response summary.** An LLM summariser writes the user-facing answer. A deterministic summary is used if the summariser is unavailable.
14. **Response assembly and rendering.** The API returns the structured response and the frontend renders summary, tags, evidence, baseline or prediction, RCA, impact, recommendations and agent flow.

7.1 Stage inputs and outputs

Stage	Main input	Main output
Guardrail	Raw query	Allowed or blocked; sanitised query
Query planning	Sanitised query, Synapt context	Query plan: filters, metrics, intent flags
Retrieval	Query plan, dataset	Ranked historical incidents with scores
Prediction (optional)	Complete telemetry	Throughput estimate, fault-occurrence estimate, fault risk tier
RCA	Ranked incidents, handoff, telemetry, predictions	Ranked hypotheses, evidence strength, gaps, uncertainty
Classification	Incidents, telemetry	P1-P4 counts, highest priority, scenario class
Impact	Telemetry or averages, predictions	Potential network and service effects, degradation indicators
Recommendation (optional)	Query, RCA, telemetry, plan, Synapt context	Procedure records, actions, status
Summary and assembly	All stage outputs	Answer, evidence, sections, trace and evaluation metadata

8. Agent Responsibility Matrix

Agent / component	Module	Calls	Returns
InputGuardrail (rule-based)	guardrail/input_gaurdrail.py	None	Allowed/blocked, sanitised query, reason
QuerySynaptAgent	agents/query_synapt_agent .py	llm_client, Synapt ask_synapt, metric_schema, Chroma, BM25	Query plan, Synapt context, ranked incidents, RCA handoff
Historical evaluation agent	agents/ (per repository)	Dataset filters, pandas	Descriptive averages labelled as baseline
NetworkPredictionAgent	agents/prediction_agent.py	prediction_config, joblib models	Throughput estimate, fault-occurrence estimate, risk tier, validity status
RCAAgent	agents/rca_agent.py	Incidents, handoff, predictions	Ranked hypotheses, supporting incidents/metrics, evidence strength, gaps
IncidentClassificationAg ent (rule-based)	agents/incident_classifier.py	Incidents, dataset-relative thresholds	P1-P4 classification, current-scenario class
NetworkImpactAnalyzer (rule-based)	agents/impact_analysis.py	Telemetry, predictions or averages	Potential impact and degradation indicators
RecommendationAgent	agents/recommendation_ag ent.py	Synapt get_synapt_procedures	Procedures, IDs and names, actions, status
TelecomAnalysisOrchest rator	agents/orchestrator.py	All agents (LangGraph)	Structured response and trace
Summariser	orchestrator stage / llm_client	LLM or deterministic fallback	User-facing answer

The incident classifier and the impact analyzer are rule-based and are labelled as such in the interface. They are not LLM agents.

9. Retrieval and Embedding Design

1. **Source.** data/telecom_network_incidents_with_id.csv is the source of truth for historical observations (100,000 rows, 27 columns per the project schema).
2. **Schema normalisation.** config/metric_schema.py maps human-readable CSV headers and aliases to stable snake_case fields so that user wording resolves to the right column.
3. **Embedding.** Incident text is embedded locally with sentence-transformers using BAAI/bge-small-en-v1.5. No embedding data leaves the machine.
4. **Vector index.** Embeddings are stored in a persistent local ChromaDB collection for semantic similarity search.
5. **Lexical index.** rank-bm25 provides keyword relevance over the same incident text.
6. **Filtering.** The query plan applies metadata filters (city, state, season, weather, incident ID, ZIP code) and numeric conditions only when the user stated them.
7. **Hybrid ranking.** Semantic similarity (approx. 60%) and BM25 relevance (approx. 40%) are combined with metric relevance scoring into one ranked list.
8. **Top-k evidence.** The highest-ranked incidents are passed to RCA and classification as evidence. The value of k is a configuration setting.

9.1 Limitations

- The historical dataset is synthetic. Retrieval quality on it does not prove behaviour on real operator data.
- Hybrid weights are tuned for the prototype and are approximate, not optimised against a labelled benchmark.
- Retrieved incidents are similar cases, not proof of cause. Moderate context precision is expected when top-k includes near neighbours (see Section 16).
- Local SOP examples are synthetic or demonstration data. Live Synapt procedure quality has not been validated.

10. Prediction Design

Prediction is optional and conditional. It runs only when the request supplies the complete required live telemetry. Historical averages are never presented as live predictions.

Item	Design
Performance target	Throughput (Mbps)
Fault target	Fault Occurrence Rate (%), mapped to a risk tier: LOW, MEDIUM, HIGH, CRITICAL
Model artifacts	ml_model/models/network_performance_model.joblib and ml_model/models/fault_tolerance_model.joblib
Model chain	The performance model runs first. Its output is used as an additional input feature to the fault tolerance model together with the telemetry.
Libraries	scikit-learn, NumPy, pandas, joblib
Configuration	config/prediction_config.py defines the feature set and groups

10.1 Feature groups

Performance, operational, load, signal, environmental, security and geography features, plus season and weather, as configured in config/prediction_config.py.

10.2 Validation and missing-feature behaviour

- Required features must all be present, numeric and finite.
- Percentage values must lie in 0 to 100. Count, user, data-usage and duration fields must be non-negative.
- Features must be compatible with the model. Missing or unavailable model artifacts and non-finite model outputs are detected.
- If any check fails, the prediction is rejected or marked incomplete. Values are never silently replaced with zero and never classified as healthy by default.
- Without live telemetry, the interface shows a **historical baseline** card, clearly labelled as descriptive averages, instead of a prediction card.

10.3 Interpretation

Outputs are estimates. Fault risk (from the fault model) is a separate quantity and must not be confused with the incident priority produced by the classifier.

11. Synapt MCP Design

Aspect	Design
Bridge	Local MCP bridge (synapt_mcp_bundle) on port 8000, endpoint http://127.0.0.1:8000/mcp, accessed with the Python mcp package and httpx/HTTP.
Domain context	ask_synapt retrieves telecom domain knowledge and schema context for the Query agent. It is advisory only. It is not historical evidence and does not prove root cause.
SOP retrieval	get_synapt_procedures is called on demand by the Recommendation agent when troubleshooting, recommendations, SOPs or resolution steps are requested. It returns procedure records, actions, IDs and names.
Recommendation status	success, no_procedures, or unavailable.
Credential flow	A registered client ID and client secret are exchanged for access, or a temporary session token is used. Credentials live in synapt_mcp_bundle/.env and are never returned to the frontend or logged.
Timeout / fallback	If the bridge is unreachable or times out, the Query agent continues without domain context and the recommendation status becomes unavailable. The response says so instead of inventing procedures.
Boundary	Port 8000 is local only. The API (8001) talks to it server-side. The browser never calls it directly.

12. API Design

Method	Endpoint	Purpose
GET	/api/health	Liveness check.
POST	/api/analyze	Full flow: guardrail, query, retrieval, optional prediction, RCA, classification, impact, optional recommendations, summary.
POST	/api/predict	Direct execution of the prediction models from supplied telemetry.
POST	/api/classify	Direct incident or current-scenario classification.
POST	/api/feedback	Store helpful / not-helpful feedback in SQLite.

> The examples below are illustrative. Field names are indicative. The authoritative contract is the set of Pydantic request models in api.py and the contract tests.

12.1 GET /api/health

```
Response 200
{ "status": "ok" }
```

12.2 POST /api/analyze

```
Request
{
  "query": "Show incidents with latency above 130 ms and suggest troubleshooting steps",
  "telemetry": null            // optional; complete object enables prediction
}

Response 200 (abridged)
{
  "answer": "...user-facing summary...",
  "historical_evidence": [ { "incident_id": "INC058190", "score": 0.0 }, ... ],
  "baseline": { ... },         // only for average / assessment queries
  "prediction": null,          // present only with valid telemetry
  "classification": { "p1": 0, "p2": 0, "p3": 0, "p4": 0, "highest": "..." },
  "rca": [ { "hypothesis": "...", "evidence_strength": "...", "gaps": ["..."] } ],
  "impact": { "potential_effects": ["..."] },
  "recommendations": { "status": "success", "procedures": [ ... ] },
  "trace": { "fallbacks": [ ... ], "evaluation": { ... } }
}
```

12.3 POST /api/predict

```
Request
{ "telemetry": { "<required feature>": 0.0, ... } }

Response 200 (valid)
{
  "status": "complete",
  "throughput_mbps": 0.0,
  "fault_occurrence_rate_pct": 0.0,
  "fault_risk": "LOW | MEDIUM | HIGH | CRITICAL",
  "note": "Predictions are estimates, not live observations."
}

Response (invalid or incomplete)
{ "status": "incomplete", "errors": ["missing feature: ...", "value out of range: ..."] }
```

12.4 POST /api/classify

```
Request
{ "telemetry": { ... } }       // or retrieved incidents, per the Pydantic model

Response 200
{ "priority": "P1 | P2 | P3 | P4", "basis": "dataset-relative prototype rule" }
```

12.5 POST /api/feedback

```
Request
{ "response_id": "...", "helpful": true, "comment": "optional text" }

Response 200
{ "stored": true }            // persisted in feedback/feedback.sqlite3
```

13. User Interface Design

The interface is a React 18.3 single-page application built with Vite 5.4. It uses no Streamlit components. Pages: Home Dashboard, AI Assistant, Network Performance and Fault Prediction, Scenario Comparison, and Evaluations.

Element	Content and design rule
Answer summary	The user-facing answer from the LLM summariser, or the deterministic summary when the summariser is unavailable.
Tags	Status chips for the query: guardrail result, evidence available, telemetry provided, recommendations requested, fallback used.
Historical evaluation	Descriptive averages over filtered records, shown when the query asks for averages, trends or assessment.
Baseline / prediction cards	Historical baseline card without telemetry. Prediction card only with valid telemetry. Each carries its trust label.
RCA	Ranked hypotheses with supporting incidents and metrics, evidence strength, and gaps. Labelled as hypotheses.
Impact	Potential network and service effects. Labelled as potential, not confirmed.
Recommended actions	Procedure-based actions with procedure ID, name and status. Labelled as requiring validation.
Agent flow	The stages that ran and the ones skipped, with fallbacks used (trace metadata).
Feedback	Helpful / not-helpful controls that call /api/feedback.
Evaluation page	Reads saved report data from evaluations/reports/ and displays model and agent evaluation summaries.

The Scenario Comparison page runs two telemetry scenarios (current and what-if) through the same prediction path and shows both side by side. It is a what-if feature, not a time trend, because the dataset has no timestamps.

14. Data Design

Primary dataset: data/telecom_network_incidents_with_id.csv. Per the project schema and context, it has 100,000 rows and 27 columns and is synthetic. Column names and aliases are defined in config/metric_schema.py. The table groups the fields by purpose.

Group	Contents	Use
Context	Incident ID and descriptive incident context	Identification, filtering, citation of evidence
Performance	Throughput (Mbps), latency, packet loss rate, call drop rate, handover success rate, jitter, connection setup success rate	Retrieval relevance, baselines, throughput target
Operations	Operational state and configuration fields	Prediction features, RCA support
Load	User counts, data usage, session duration	Prediction features, impact analysis
Signal	Signal-quality fields	Prediction features
Environment	Season and weather	Filters and prediction features
Security	Security-related fields	Prediction features
Geography	City, state, ZIP code	Filters and prediction features
Fault target	Fault Occurrence Rate (%)	Fault model target and risk tier

Supporting context files (domain, incident, SOP and schema) are in files_load_context_files/. The exact 27 column names are taken from config/metric_schema.py and are not reproduced here.

15. Security Design

Topic	Design
Secrets in .env	The AICredits key is in the root .env. Synapt provider and agent credentials are in synapt_mcp_bundle/.env. Only variable names appear in documentation.
No committed credentials	Credentials must not be committed. Both .env files belong in .gitignore. Rotate any key that was ever committed or shared.
Credential handling	Keys are read server-side by python-dotenv. They are never returned in API responses, logged, or sent to the browser.
Synapt credential flow	Client ID and secret exchange or a temporary session token, handled by the bridge.
Localhost binding	Services bind to 127.0.0.1 (8000, 8001, 8790) and are not exposed externally by default.
TLS verification	Outbound HTTPS calls (AICredits and any remote Synapt provider) keep certificate verification enabled.
CORS	The API should allow only the local frontend origin (http://127.0.0.1:8790). Confirm in api.py.
Input validation	Pydantic models plus InputGuardrail: empty and length checks, prompt-injection, system-prompt and secret extraction, database/code/command injection, and telecom-domain relevance.
Output safety	Responses carry trust labels so estimates and hypotheses are not read as facts.

16. Evaluation Design

16.1 Objectives and judge

- Check that retrieval returns relevant, filter-compliant evidence, that RCA is faithful and cautious, and that the final answer is relevant and grounded.
- **LLM-as-judge provider:** AICredits OpenAI-compatible endpoint with google/gemini-3.1-flash-lite.
- **Tools:** DeepEval (agent-specific checks) and RAGAS (Synapt/RAG path). Reports are saved as JSON in evaluations/reports/ and read by the Evaluations page.

16.2 Metrics

Stage	Metrics
Retrieval (DeepEval)	Contextual Precision, Contextual Recall, Contextual Relevancy, Retrieval Filter Compliance
RCA (DeepEval)	RCA Quality (GEval), Faithfulness, RCA Evidence and Caution
Final answer (DeepEval)	Answer Relevancy, Faithfulness, Hallucination
RAG path (RAGAS)	Faithfulness, Context Precision, Context Relevance
Models	R-squared, MAE and RMSE on held-out test data
Contracts	Prediction-agent contract checks and API contract tests

Pass thresholds are set in the evaluation configuration and are not restated here.

16.3 Current results

ML models (trained on 80,000 samples, evaluated on 20,000 held-out test samples):

Model	R-squared	MAE	RMSE
Network performance (throughput)	0.9782	29.44 Mbps	35.42 Mbps
Fault risk (fault occurrence rate)	0.9907	1.07 %	1.35 %

RAG retrieval quality: 0.96. **DeepEval example case** (test case latency_above_130):

Stage	Metric	Score
Retrieval	Contextual Precision	1.00
Retrieval	Contextual Recall	1.00
Retrieval	Contextual Relevancy	1.00
Retrieval	Retrieval Filter Compliance	1.00
RCA	RCA Quality (GEval)	1.00
RCA	Faithfulness	1.00
RCA	RCA Evidence and Caution	1.00
Final answer	Answer Relevancy	1.00
Final answer	Faithfulness	0.86
Final answer	Hallucination	1.00

The final-answer faithfulness score of 0.86 reflects one defect: the output stated 165 historical incidents while the retrieval context listed only 5. RAGAS report values are saved in evaluations/reports/ and are not reproduced in this document.

16.4 Limitations and interpretation

- ML metrics are measured on a held-out split of the same synthetic dataset. They show fit to that dataset, not live network accuracy.
- The DeepEval figures above come from one test case. They show the pipeline works on that case and should not be generalised.
- LLM judges can vary between runs and models. Scores are indicative.
- **Moderate Context Precision** means some retrieved contexts are less relevant or ranked lower than ideal. This is expected when top-k includes near neighbours and the dataset is synthetic with many similar records. It does not mean the answer is wrong, but it is the first place to tune filters, k and hybrid weights.

17. Failure Handling and Fallback Matrix

Failure	Detection	Behaviour	User-visible result
LLM unavailable (planner)	Timeout or error from llm_client	Deterministic fallback parser builds the query plan	Analysis continues; trace notes the fallback
LLM unavailable (summariser)	Timeout or error	Deterministic summary from structured sections	Plain summary; trace notes the fallback
Synapt unavailable	MCP connection or timeout error	Continue without domain context	Answer notes missing context
Missing telemetry	Required features absent	Skip prediction. Offer historical baseline if averages were requested	Baseline card, no prediction card
Invalid telemetry	Range, type or finiteness check fails	Reject or mark incomplete	Error list; no silent zeros
Missing historical evidence	Empty ranked list after filters	RCA reports insufficient evidence	States the gap; no invented cause
SOP retrieval failure	Empty result or bridge error	Status no_procedures or unavailable	No procedures shown; status explained
Prediction model failure	Missing artifact or non-finite output	Mark prediction unavailable	Message; rest of the analysis continues
Judge timeout (evaluation)	Timeout from judge call	Metric recorded as unscored and rerun	Report shows the case as unscored

18. Deployment and Runbook

The system runs as three local services. The commands below are those recorded in the project's application-flow brief.

1. **Configure.** Create the root .env (AICredits settings) and synapt_mcp_bundle/.env (Synapt credentials). Do not commit either.
2. **Start the Synapt MCP bridge** from synapt_mcp_bundle on port 8000 using the bundle's launch script.
3. **Start the API** on port 8001.
4. **Start the frontend** on port 8790.

```
# API (Windows PowerShell)
.\venv\Scripts\python.exe -m uvicorn api:app --reload --port 8001

# Frontend
python -m http.server 8790 --bind 127.0.0.1 --directory frontend

# Open
http://127.0.0.1:8790
```

18.1 Health checks

Service	Check	Expected
API (8001)	GET http://127.0.0.1:8001/api/health	HTTP 200 with a status field
Synapt MCP (8000)	Connect to http://127.0.0.1:8000/mcp	MCP endpoint responds
Frontend (8790)	Open http://127.0.0.1:8790	Login and assistant page load
End to end	Ask a sample query in the AI Assistant	Answer with evidence and trace

19. Risks, Limitations and Future Improvements

Risk / limitation	Impact	Mitigation or future work
Synthetic historical and SOP data	Results may not transfer to real networks	Validate on operator data. Connect and validate a live SOP source.
No live telemetry integration	Predictions rely on manually supplied inputs	Add a real-time telemetry connector with monitoring.
Prediction metrics on held-out synthetic split	Overstates real-world accuracy	Backtest on real data. Add drift monitoring and retraining.
LLM variability and dependency	Planning, summary and judge outputs can vary	Deterministic fallbacks exist. Add caching and regression tests.
Moderate context precision	Some irrelevant context reaches RCA	Tune k, filters and hybrid weights. Add a re-ranker.
Synapt availability and content quality	No context or procedures	Fallback statuses exist. Validate live procedure quality.
Prototype classification thresholds	Not operator SLAs	Replace with operator-approved thresholds.
Single-case DeepEval example	Limited coverage	Broaden the evaluation set and track results over time.

20. Appendix

A. Repository file map

Path	Purpose
api.py	Starlette API, Pydantic models, endpoints
agents/orchestrator.py	TelecomAnalysisOrchestrator, LangGraph workflow
agents/query_synapt_agent.py	Query/Synapt agent, planning, retrieval
agents/prediction_agent.py	Throughput and fault prediction
agents/rca_agent.py	RCA hypotheses
agents/incident_classifier.py	Incident classification
agents/impact_analysis.py	Network impact analyzer
agents/recommendation_agent.py	SOP retrieval and recommendations
guardrail/input_gaurdrail.py	Input guardrail
llm/llm_client.py	LLM provider client
config/metric_schema.py	Canonical metric schema and aliases
config/prediction_config.py	Prediction features and settings
data/telecom_network_incidents_with_id.csv	Historical incident dataset
ml_model/models/	network_performance_model.joblib, fault_tolerance_model.joblib
files_load_context_files/	Domain, incident, SOP and schema context files
synapt_mcp_bundle/	Local Synapt MCP bridge and its .env
evaluations/	DeepEval, RAGAS, contract and model evaluation; reports/ holds saved JSON
feedback/feedback.sqlite3	SQLite feedback store
frontend/src/	React application; pages/Evaluations.jsx
README.md, APPLICATION_FLOW.md, ARCHITECTURE_AND_PPT_GUIDE.md, PROJECT_NAVIGATION.md	Project documentation

B. Environment variables (names only)

Variable / group	Purpose
DEEPEVAL_MODEL	Judge model for DeepEval (current: google/gemini-3.1-flash-lite)
AICredits API key, base URL, model	Active LLM provider settings (names as defined in .env)
Synapt client ID, client secret, session token	Synapt MCP authentication (in synapt_mcp_bundle/.env)
OpenRouter and Claude settings	Legacy compatibility; not the active provider
Timeout, temperature, token limit, retries	LLM client behaviour (llm/llm_client.py)

Secret values are never shown in this document or committed to the repository.

C. Sample queries

- Show incidents with latency above 130 ms.
- Find incidents in a given city and season with degraded performance.
- What is the average throughput for incidents in a given state?
- Why did packet loss rise for these incidents, and what should I check first? (RCA and recommendations)

- Here is current telemetry. Estimate throughput and fault risk. (live prediction)

D. Glossary

Term	Meaning
RCA	Root-cause analysis. Here, a ranked, evidence-linked hypothesis.
SOP	Standard operating procedure for troubleshooting.
MCP	Model Context Protocol, used to connect to Synapt.
Synapt	External context service providing domain/schema context and SOPs (Synapt Context Substrate).
RAG	Retrieval-augmented generation.
BM25	Lexical ranking function for keyword search.
ChromaDB	Local vector database for semantic search.
BGE	BAAI/bge-small-en-v1.5 embedding model.
LangGraph	Framework for stateful multi-step agent workflows.
DeepEval / RAGAS	Evaluation frameworks for LLM and RAG outputs.
P1-P4	Incident priority levels from the classifier.
Telemetry	Current network measurements supplied by the user.
Baseline	Historical average, not a live reading or forecast.

21. References and Source Files

- Telecom Network Operations Assistant architecture brief (project architecture and application flow).
- Technical Stack document (runtime, LLM, retrieval, ML, Synapt, evaluation).
- Application Design Document prompt and project facts (ports, models, dataset size).
- Architecture diagram: telecom_architecture.png (Figure 1).
- Evaluation reports: evaluations/reports/ and the evaluation results supplied for the ML models and the DeepEval test case.
- Source repository files listed in Appendix A.